

şimdi de senden yine asenkron bir başka yapıyı implement etmeni isteyeceğim. Rsa şifreleme algoritmasında a üzeri b işlemi var. Burada hem a, hem de b sayısını ikilik sayı sisteminde düşünerek işlem yapılan multiply-add algoritması var. B sayısının ikilik düzendeki gösteriminde karşına çıkan bit 0 ise multiply yapıyorsun, 1 ise hem multiply, hem add yapıyorsun. Vhdl ile bu işlemi de implement edebilir misin?

Şimdi bu görevleri yerine getirmem gerekiyor. Macbook m2 işlemcili bilgisayar kullanıyorum. Bu prjeyi yapmam ve anlamam gereken konular nelerdir bana öğrenmem konuları önce açıkla nerelerden daha derinlemesine öğrenebilirim referans ver. ardından yapmaya başla. VHDL i vs code da yazıyorum ghdl ile terminalden derliyorum. ardından VS Code WaveTrace dan sinyali inceliyorum.

Ön Bilgi

RSA şifrelemede bahsettiğin işlem **Modular Exponentiation** (Modüler Üs Alma) işlemidir:

$$R = A^B \pmod{N}.$$

Senin tarif ettiğin "0 ise multiply, 1 ise multiply-add" algoritması aslında **"Square and Multiply"** (Kare Al ve Çarp) algoritmasıdır. Bir sayının üssünü alırken (üs binary'ye çevrildiğinde):

- Bit **0** ise: Sadece sayının karesini alırsın (**Square**).
- Bit **1** ise: Sayının karesini alır (**Square**) VE taban sayı ile çarparsın (**Multiply**).

Bu proje, önceki LFSR örneğine göre biraz daha karmaşıktır çünkü bir **Durum Makinesi (Finite State Machine - FSM)** gerektirir.

Önce öğrenmen gereken kavramları netleştirelim, sonra kodlamaya geçelim.

1. Öğrenmen Gereken Konular ve Referanslar

Bu projeyi tam anlamıyla kavramak için şu üç temel yapı taşını bilmen gerekir:

A. Modüler Aritmetik ve RSA Matematiği

RSA algoritmasının kalbi şudur: $C = M^e \pmod{n}$. Burada çok büyük sayıların üssünü alıp bir modül (n) sayısına böleriz.

- **Ne öğrenmelisin?** Modüler aritmetik nedir? $A \times B \pmod{N}$ işlemi bilgisayarda nasıl yapılır?
- **Neden gerekli?** Doğrudan A^B işlemini yapamazsın çünkü sayı bit genişliğine sığmayacak kadar devasa olur (taşma/overflow). Bu yüzden her çarpma işleminden sonra hemen modunu alman gerekir.
- **Referans:** Khan Academy - "Modular Arithmetic" serisi veya Christof Paar'ın "Understanding Cryptography" YouTube serisi (Lecture 1-2).

B. Square and Multiply Algoritması

Senin bahsettiğin ikilik sistem (binary) tarama yöntemidir. Üs sayısını (B) bit bit tarayarak işlemi hızlandırır.

- **Algoritma:**

1. Sonuç = 1 yap.
2. B sayısının en anlamlı bitinden (MSB) en anlamsız bitine (LSB) doğru git.
3. Her adımda: **Sonuç = (Sonuç * Sonuç) mod N** (Kare Al).
4. Eğer o sıradaki bit '1' ise: **Sonuç = (Sonuç * A) mod N** (Çarp).

- **Görsel:**

C. VHDL'de Finite State Machine (FSM)

Bu işlem tek bir saat darbesinde (clock cycle) yapılamaz. Bir döngüye ihtiyacın var.

- **Ne öğrenmelisin?** IDLE (bekle), CALC (hesapla), DONE (bitti) gibi durumlar arasında geçiş yapan bir yapı (State Machine) nasıl kurulur?
- **Önemi:** Donanımda "for döngüsü" yazılım dillerindeki gibi çalışmaz. Döngüleri bir FSM ile zaman içinde yönetiriz.
- **Referans:** "VHDL FSM Design" araması ile Nandland.com sitesindeki FSM örneklerine bakabilirsin.

Neden Yapıyoruz?

Yaptığın bu işlem, internetteki güvenliğin (HTTPS, bankacılık, WhatsApp mesajlaşmaları) temel taşı olan **RSA Şifrelemesinin** kalbidir.

Bunu "**Tek Yönlü Matematik**" (One-Way Function) olarak düşünebiliriz.

Boya Karıştırma Analojisi:

- Elimizde iki renk boya olsun: **Sarı** ve **Mavi**.
- Bunları karıştırırsan **Yeşil** elde edersin. Bu işlem çok kolaydır (Çarpma işlemi).
- Ama sana bir kova **Yeşil** boya versem ve "Hadi bunu tekrar Sarı ve Mavi olarak ayır" desem? Bu neredeyse imkansızdır.

İşte **Modular Exponentiation** ($A^B \pmod{N}$) işlemi matematikteki "boya karıştırma" işlemidir.

- **İleriye gitmek kolaydır:** Sayıları çarpar, modunu alırsın (Senin VHDL kodunun yaptığı iş).
- **Geriye dönmek imkansızdır:** Sonucu (445) bilen bir hacker, hangi sayıların çarpıldığını (4 ve 13) makul bir sürede bulamaz.

Senaryo:

1. **Mesaj:** Diyelim ki "Merhaba" mesajını bir sayıya çevirdik: **4**.
2. **Anahtar (Şifre):** Şifrelemek için kullanılan gizli sayı: **13**.
3. **Kilit (Mod):** Herkesin bildiği ortak sayı: **497**.

Senin yazdığın kod, 4 sayısını alıp, 13 ve 497 ile o kadar karmaşık bir matematiksel blenderdan geçirdi ki, ortaya 445 çıktı.

Yolda bu mesajı yakalayan bir hacker sadece 445 görür. Elinde "13" sayısı (anahtar) olmadığı sürece bu 445'i tekrar 4'e (Merhaba) çeviremez.

2. Neden Donanım (VHDL) ile Yapıyoruz?

Bilgisayardaki yazılımlar (Python, C) bu işlemi yapabilir. Ama biz VHDL ile donanım tasarlıyoruz. Neden?

1. **Hız:** Gerçek hayatta biz 4 ve 13 gibi küçük sayılar kullanmayız. **1024 basamaklı** devasa sayılar (binary olarak) kullanırız. Normal bir işlemci bu sayıları çarparken zorlanır ve yavaşlar. Senin tasarladığın bu devre, **sadece ve sadece bu işi yapmak için** özelleşmiş bir "Matematik Canavarı"dır. Çok daha hızlı şifreleme yapar.
2. **Güvenlik:** Donanım seviyesinde çalışan şifreleme modülleri, yazılıma göre "hacklenmesi" daha zordur. İşletim sistemi virüs kapşa bile FPGA içindeki bu devre izole çalışır.

Kodu çalıştırma 16 bit

```
orhunboydak@Orhuns-MacBook-Air ~ % cd Desktop
```

```
orhunboydak@Orhuns-MacBook-Air Desktop % cd rsa_proje
```

```
orhunboydak@Orhuns-MacBook-Air rsa_proje % ghdl -a mod_exp.vhd mod_exp_tb.vhd
```

```
ghdl -e mod_exp_tb
```

```
ghdl -r mod_exp_tb --vcd=dalga.vcd
```

1. Hazırlık: WaveTrace Ayarları

Analize başlamadan önce WaveTrace'de şu ayarları yaparsan hayatın kolaylaşır:

1. **Sinyalleri Ekle:** `clk`, `state`, `bit_count`, `r_result`, `r_exp` (veya `exp`), `done`.
2. **Format Değiştir:** `r_result`, `base`, `modulus` sinyallerine sağ tıkla -> **Radix** -> **Unsigned Decimal**. (Böylece `10111101` yerine `445` görürsün).
3. **Zoom:** Simülasyonun en başına değil, `start` sinyalinin '1' olduğu yere odaklan.

2. Dalga Formu Analizi (Adım Adım)

Üs sayımız (Exponent) 13.

Binary karşılığı (16 bit): 0000 0000 0000 1101.

Algoritmamız en soldaki bittten (15. bit) başlayıp sağa (0. bite) doğru tarıyor.

Bölüm A: Sessiz Dönem (Leading Zeros)

`bit_count` 15'ten 4'e kadar inerken (Bitlerin hepsi '0'):

- **State:** `SQUARE` -> `MULTIPLY` arasında gidip gelir.
- **İşlem:** `r_result` başlangıçta 1.
 - **Square:** $1 \times 1 = 1$.
 - **Multiply:** Bit '0' olduğu için çarpma (Multiply) yapılmaz veya etkisizdir.
- **Waveform'da Ne Görürsün?**
 - `state` sürekli değişir.
 - `bit_count` azalır (15, 14, ..., 4).

- Ama `r_result` hep "1" olarak kalır. (Burası bozuk değil, doğrusu budur. 1'in karesi hep 1'dir).

Bölüm B: Aksiyon Başlıyor (Bit 3: '1')

Binary: ...1101 (Kalın olan bit)

- **SQUARE State:** `r_result` (1) karesi alınır 1.
- **MULTIPLY State:** Bit '1' olduğu için `r_result` (1) ile `base` (4) çarpılır.
- **Waveform:**
 - `r_result` değeri 1'den 4'e yükselir.

Bölüm C: İkinci '1' (Bit 2: '1')

Binary: ...1101

- **SQUARE State:** Önceki sonuç (4) karesi alınır. $4 \times 4 = 16$.
 - $16 \pmod{47} = 16$. -> `r_result` **16** olur.
- **MULTIPLY State:** Bit '1' olduğu için, sonuç (16) ile `base` (4) çarpılır.
 - $16 \times 4 = 64$.
 - $64 \pmod{47} = 17$. -> `r_result` **17** olur.

Bölüm D: '0' Biti (Bit 1: '0')

Binary: ...1101

- **SQUARE State:** Önceki sonuç (64) karesi alınır.
 - $64 \times 64 = 4096$.
 - Mod işlemi: $4096 \pmod{47} = 12$.
 - *Hesap:* $47 \times 8 = 376$. $4096 - 376 = 3720$.
 - **Waveform:** `r_result` 64'ten **120**'ye güncellenir.
- **MULTIPLY State:** Bit '0' olduğu için çarpma yapılmaz!
 - **Waveform:** `r_result` **120** olarak kalır (Değişmez).

Bölüm E: Son '1' Biti (Bit 0: '1')

Binary: ...1101 (LSB - Son Bit)

- **SQUARE State:** Önceki sonuç (120) karesi alınır.
 - $12 \times 12 = 144$.
 - Mod işlemi: $144 \pmod{47} = 484$.
 - **Waveform:** `r_result` 120'den **484**'e değişir.
- **MULTIPLY State:** Bit '1' olduğu için sonuç (484) ile `base` (4) çarpılır.
 - $484 \times 4 = 1936$.
 - Mod işlemi: $1936 \pmod{47} = 445$.
 - *Hesap:* $47 \times 3 = 141$. $1936 - 141 = 1795$.
 - **Waveform:** `r_result` 484'ten **445**'e değişir.

Bölüm F: Bitiş (FINISH)

- `bit_count` 0 olduğu için FSM `FINISH` durumuna geçer.
- **Waveform:** `done` sinyali '0'dan '1'e çıkar. `r_result` **445** değerinde sabitlenir.

32 bit

Nasıl çalışır

orhunboydak@Orhuns-MacBook-Air ~ % cd Desktop

orhunboydak@Orhuns-MacBook-Air Desktop % cd rsa_proje_32

orhunboydak@Orhuns-MacBook-Air rsa_proje_32 % ghdl -a mod_exp.vhd mod_exp_tb.vhd

ghdl -e mod_exp_tb

ghdl -r mod_exp_tb --vcd=dalga_32bit.vcd

mod_exp_tb.vhd:88:13:@705ns:(report note): BASARILI: 32-bit RSA islemi dogru calisti!

mod_exp_tb.vhd:94:9:@705ns:(assertion failure): Simülasyon Tamamlandı.

./mod_exp_tb:error: assertion failed

instance: .mod_exp_tb(behavioral).P1

./mod_exp_tb:error: simulation failed

Sıradaki Adım: Profesyoneller Nasıl Yapar?

Stajında veya bitirme projende seni öne geçirecek "gerçek mühendislik" adımı şudur: **Montgomery Çarpımı (Montgomery Multiplication)**.

Bu yöntem, bölme işlemini (mod alma) tamamen ortadan kaldırır ve onun yerine donanımın çok sevdiği **Toplama ve Bit Kaydırma (Shift)** işlemlerini kullanır.

Analojisi şudur:

- **Şu anki yöntem:** İki sayıyı çarpıyorsun, sonuç çok büyüyor (dağ gibi), sonra onu modül sayısına bölerek küçültüyorsun (dağı tıraşlıyorsun). Bu çok yorucu.
- **Montgomery Yöntemi:** Sayıları sihirli bir aynadan geçiriyorsun (Montgomery Domain). Bu aynada çarpma işlemi yaparken sayılar hiç büyümüyor, hep küçük kalıyor. İş bitince aynadan geri çıkarıyorsun.

Bölme yapmadan mod alan (Montgomery) Yapı

1. Temel Problem: Bilgisayarlar Bölmeyi Sevmez

Senin şu anki kodunda şöyle bir satır var:

```
result <= (a * b) mod N;
```

Donanım (FPGA/ASIC) için bu satır bir kabustur.

- Çarpma:** $(A \times B)$ sonucu çok büyük (örneğin $1024 \text{ bit} \times 1024 \text{ bit} = 2048 \text{ bit}$).
- Bölme:** Sonra bu dev sayıyı N 'ye bölüp kalanı bulman gerekir. Bölme işlemi donanımda çok fazla lojik kapı harcar ve çok yavaştır (çünkü bir önceki basamağın sonucunu beklemeden sonrakini hesaplayamazsın).

Hedef: Öyle bir yöntem bulalım ki, hiç **bölme işlemi yapmadan** mod alabilelim.

2. Çözüm: Montgomery Dünyası (Parallel Universe)

Montgomery algoritması, sayıları "Normal Dünya"dan alıp "Montgomery Dünyası"na (Domain) götürür.

- Normal Dünya:** Mod almak için N 'ye bölmek zorundasın. Zordur.
- Montgomery Dünyası:** Mod almak için R sayısına bölmek yeterlidir. Eğer R sayısını **2'nin kuvveti** (örneğin 2^{32}) seçersen, bilgisayar için "bölmek" demek, sadece **bitleri sağa kaydırmak (shift right)** veya **alt bitleri çöpe atmak** demektir. Bu işlem donanımda **BEDAVADIR**.

Analoji: Kumarhane Fişleri (Casino Chips)

- Giriş (Dönüşüm):** Cebinde Türk Lirası (Normal Sayı) var. Kumarhaneye giriyorsun. Lira ile oyun oynayamazsın. Kasaya gidip parayı **Fişe (Montgomery Sayısı)** çeviriyorsun. Bu işlem biraz maliyetlidir (çarpma gerektirir).
- Oyun (İşlem):** İçeride yüzlerce oyun oynuyorsun (Üs alma işlemi). Fişleri birbiriyle çarpıyorsun, topluyorsun. Burada işlemler çok hızlı çünkü kasa sadece fiş kabul ediyor, para üstü hesaplamak (mod almak) çok kolay.
- Çıkış (Ters Dönüşüm):** Oyun bitti. Elindeki fişleri kasaya götürüp tekrar Türk Lirasına çeviriyorsun.

RSA Bağlamı:

RSA'da binlerce kez çarpma işlemi (A^1 gibi) yaparız.

- Sayıları **bir kere** Montgomery dünyasına çeviririz.
- Binlerce çarpma işlemini orada "bölmesiz" (hızlıca) yaparız.
- En sonda **bir kere** normal dünyaya döneriz.

3. Matematiksel Hile (Nasıl Çalışıyor?)

Normalde $a \times b \pmod N$ işlemini arıyoruz.

Montgomery yönteminde bir R sayısı seçiyoruz (Genelde $R = 2^{B_{\text{tag}}}$).

Sayımız A ise, onun Montgomery karşılığı:

$$A = A \times R \pmod N$$

Montgomery Çarpımı (MonPro) fonksiyonu bize şunu verir:

$$\text{MonPro}(A, B) = A \times B \times R^{-1} \pmod N$$

Bu formülün detayına boğulmana gerek yok. Sihir şurada gizli: Algoritma bu işlemi yaparken **asla N sayısına bölme yapmaz**. Sadece toplama ve 2'ye bölme (bit kaydırma) yapar.

4. Algoritmanın Adımları (VHDL İçin Yol Haritası)

Kodlayacağımız algoritma tam olarak şu adımları izleyecek. Diyelim ki a ve b sayılarını çarpacağız (M bizim modülümüz):

Montgomery Multiplication (MonPro) Algoritması:

1. Sonuç $result = 0$ yap.
2. sayısının her bir biti (x) için (0'dan 31'e kadar) döngü kur:
 - Eğer $x = 1$ ise, $result = result + a$ yap.
 - Bu toplama işleminden sonra sayı tek mi çift mi bak (LSB kontrolü).
 - Eğer sayı tek ise: $result = result + M$ (Modülü ekle ki sayı çift olsun).
 - $result = result / 2$ (Sağa 1 bit kaydır). **İşte sihir burada! Bölme yok, shift var.**
3. Eğer $a > M$ ise, $a = a - M$ yap.

Fark Ettin mi?

Yukarıdaki algoritmada hiç $/ N$ veya $\% N$ operatörü yok. Sadece:

- Toplama ($+$)
- Bit kontrolü (`if bit = '1'`)
- Sağa kaydırma (`shift right / div 2`)
- Çıkarma ($-$)

Bu işlemler FPGA üzerinde inanılmaz hızlı çalışır ve az yer kaplar.

Kod nasıl çalışır

```
orhunboydak@Orhuns-MacBook-Air ~ % cd Desktop
```

```
orhunboydak@Orhuns-MacBook-Air Desktop % cd rsa_proje_montgomery
```

```
orhunboydak@Orhuns-MacBook-Air rsa_proje_montgomery % ghdl -a mon_pro.vhd mon_pro_tb.vhd
```

```
ghdl -e mon_pro_tb
```

```
ghdl -r mon_pro_tb --vcd=dalga_montgomery.vcd
```

Dalga Formunda (WaveTrace) Neye Bakacaksın?

1. **Bit Sayacı:** `bit_cnt` sinyalinin 0'dan 31'e kadar saydığını gör. Bu, döngünün çalıştığını gösterir.
2. **Akümülatör (`r_acc`):** `ADD_SHIFT` aşamasında bu değerin sürekli değiştiğini, bazen arttığını (toplama) bazen azaldığını (shift/bölme) göreceksin.
3. **Kaydırma:** En önemlisi `v_sum srl 1` mantığıdır. Sinyal değerlerinin ikilik tabanda sürekli sağa kaydığını (bölündüğünü) hayal et.
4. **Bölme Yok:** Kodun içinde `/` operatörünün olmadığını, sadece `srl` (shift right logical) olduğunu teyit et.

Yeni Görev: "Motor Değişimi" (Engine Swap)

Eski `mod_exp.vhd` dosyamızda çarpma işlemini şöyle yapıyorduk: `r_result <= (r_result * r_result) mod r_mod;` (Yazılımsal, yavaş ve alan kaplayan yöntem)

Şimdi bu satırları sileceğiz. Yerine senin az önce doğruladığın `mon_pro` donanımını çağırıp iş yaptıracacağız.

Bunun için `mod_exp` modülünün **Orkestra Şefi** gibi davranması gerekiyor. Şöyle bir diyalog dönecek:

- **Orkestra Şefi (Mod Exp):** "Hey Çarpıcı (MonPro), sana iki sayı veriyorum, bunları çarp."
- **Şef:** (Bekler...)
- **Çarpıcı:** "İşim bitti, al sonucu."
- **Şef:** "Tamam, şimdi sıradaki işlemi yapalım."

Bu yapıya **"Master-Slave"** kontrolü denir. Bunu yönetmek için State Machine'imiz (FSM) biraz daha detaylanacak.

Nasıl Çalışır

Ne Değişti? (Öğrenmen Gereken Kritik Konsept)

Kodun içine dikkatli bakarsan şunu göreceksin:

- Artık matematiksel operatör (`*` , `mod`) yok.
- Bunun yerine **sinyal atama** (`mp_a <= ...`) ve **bekleme mantığı** (`if mp_done = '1'`) var.

Bu, bir işlemcinin (CPU) çalışma mantığına çok benzer. CPU da "ALU" (Arithmetic Logic Unit) denilen alt birime sayıları gönderir, "Topla" sinyali verir ve sonucu bekler. Sen şu an **kendi küçük RSA işlemcini tasarladın**.

Nasıl Test Edeceğiz?

`mod_exp_tb.vhd` dosyan zaten hazırды (32 bitlik olan). Ona dokunmana gerek yok. Sadece terminalden tekrar `mod_exp` 'i derlemen yeterli.

Ancak unutma: `mon_pro.vhd` dosyasının da derlenmesi lazım çünkü `mod_exp` artık onu kullanıyor.

1. Önce alt parçayı derle (Engine)
`ghdl -a mon_pro.vhd`
2. Sonra ana parçayı derle (Car)
`ghdl -a mod_exp.vhd`
3. Testbench'i derle
`ghdl -a mod_exp_tb.vhd`
4. İnşa et ve Çalıştır
`ghdl -e mod_exp_tb`
`ghdl -r mod_exp_tb --vcd=dalga_entegre.vcd`

Neden Başarılı?

1. **Zamanlama:** `done` sinyali artık çok daha geç geliyor (simülasyon süresi uzadı). Bu, ana modülün (`mod_exp`), alt modülü (`mon_pro`) her bit için tek tek çağırdığını ve cevabını beklediğini kanıtlıyor.

2. **Handshake (El Sıkışma):** `done` sinyali gelmeden hemen önce `result` hattında bir hareketlilik var. Bu, son işlemin yapılıp sonucun yazıldığını gösteriyor.

Şu an elinde **mekanik olarak kusursuz çalışan** bir sistem var. Bir Savunma Sanayi stajyeri olarak gurur duyabilirsin: Bir "System Integration" (Sistem Entegrasyonu) yaptın.

Ancak... Küçük Bir "Matematiksel" Sır

Eğer WaveTrace'de çıkan sonuca (`result`) sağ tıklayıp Hex değerine bakarsan, muhtemelen **3347A510** (Python ile bulduğumuz doğru sonuç) olmadığını göreceksin.

"Kod bozuk mu?"

Hayır, kodun sağlam. Sorun şu: Dil farkı.

Hatırlarsan Montgomery için "Kumarhane Fişi" analojisini kullanmıştık.

- Sen şu an hesaplamayı yaptın ama sonuç hala **"Fiş"** (Montgomery Domain) cinsinden.
- Bizim bunu harcayabilmemiz için tekrar **"TL"** (Normal Domain) cinsine çevirmemiz lazım.

Son Dokunuş: Final Dönüşümü (The Final Trick)

Montgomery algoritmasının en zekice kısmı burasıdır. Sonucu normale çevirmek için karmaşık bir işlem yapmana gerek yok.

Formül şudur:

$$Normaon = MontgomeryCarpM(Montomeryon, 1)$$

Yani; elimizdeki sonucu, **sadece "1" sayısı ile** senin o harika `mon_pro` modülüne sokarsak, o bize temizlenmiş, gerçek sonucu verir. Donanımcılar buna "Post-Computation" der.

Görev: FSM'e "Final Dönüşüm" Ekleme

Mevcut `mod_exp.vhd` dosyasındaki State Machine'e (FSM) sadece küçük bir adım ekleyeceğiz. İşlem bitince (`FINISH` 'ten önce), sonucu 1 ile çarpıp öyle bitireceğiz.

Bunu yapmaya hazır mısın? Bu adımı da eklersek, simülasyon sonucunda o meşhur **3347A510** değerini göreceğiz ve proje tam anlamıyla "Road Legal" (trafiğe çıkmaya hazır) olacak.

Analoji: Kumarhaneden çıkarken (Bit döngüsü bittiğinde), elimizdeki fişleri kasaya götürüp "Bunları TL'ye çevir" (Convert to Normal Domain) diyeceğiz.

Bunun için FSM (Durum Makinesi) akışımıza iki yeni durak ekliyoruz:

- `START_FINAL` : Çevirme işlemini başlat (Montgomery Çarpıcısına "Sonuç" ve "1" sayısını ver).
- `WAIT_FINAL` : İşlemin bitmesini bekle ve sonucu kaydet.

Nasıl Çalışır

```
cd Desktop
cd rsa_proje_final
ghdl -a mon_pro.vhd
ghdl -a mod_exp.vhd
ghdl -a mod_exp_tb.vhd
ghdl -e mod_exp_tb
ghdl -r mod_exp_tb --vcd=final_zafer.vcd
```

Projenin Özeti: Donanım Tabanlı RSA Hızlandırıcısı

Bu projede, güvenli iletişimin temeli olan RSA şifreleme algoritmasının en kritik işlemi olan **Modüler Üs Alma** ($A^B \pmod{N}$) işlemini gerçekleştiren özelleşmiş bir donanım tasarladık.

1. Aşama: Temel Matematik (Square and Multiply)

İlk olarak, "Bilgisayar A^B işlemini nasıl yapar?" sorusuna cevap aradık.

- Sorun:** A^B gibi devasa bir sayıyı hesaplayıp sonra N 'ye bölmek bilgisayar (ve donanım) için imkansızdır. Sayılar hafızaya sığmaz.
- Teorem/Yöntem:** **Square and Multiply (Kare Al ve Çarp)** algoritması.
- Mantık:** Üs sayısını (B) ikilik (binary) tabanda taradık.
 - Bit 0 ise: Sadece karesini al ($= A^2 \pmod{N}$).
 - Bit 1 ise: Karesini al VE tabanla çarp ($= (A^2 \times A) \pmod{N}$).
- Sonuç:** Bu yöntemle, 1024 bitlik bir üs için sadece yaklaşık 1.500-2.000 işlem yaparak sonucu bulabileceğimizi gördük.
- İlk Tasarım:** Standart çarpma ($*$) ve mod alma ($\pmod{N}$) operatörlerini kullanan 32-bitlik bir prototip yaptık.

2. Aşama: Donanım Darboğazı ve Montgomery Çarpımı

İlk tasarımı analiz ettiğimizde, gerçek dünyada (FPGA/ASIC) çalışamayacak bir sorun fark ettik.

- Sorun:** $X \pmod{N}$ işlemi (bölme), donanımda çok maliyetlidir (çok alan kaplar, çok yavaştır). Her çarpma işleminden sonra bölme yapmak sistemi kilitler.
- Çözüm:** **Montgomery Multiplication (Montgomery Çarpımı)**.
- Formül:** $MonPro(A, B) = A \times B \times R^{-1} \pmod{N}$.
- Neden Yaptık?** Montgomery algoritması, bölme işlemini tamamen ortadan kaldırır. Bölme yerine sadece **toplama** ve **sağa kaydırma (shift)** kullanır. Donanım, kaydırma işlemini "bedavaya" yapar.
- Fark:** Önceki tasarımda pahalı "Bölme Motoru" varken, bunu söküp yerine çok hızlı ve ucuz "Kaydırma Motoru" (mon_pro modülü) taktık.

3. Aşama: Alt Modül Tasarımı (Bit-Serial Montgomery)

Montgomery çarpımını donanımda nasıl yapacağımızı tasarladık.

- **Yöntem: Bit-Serial (Bit Bit İşleme)** yaklaşımı.
- **Mantık:** Çarpılan sayının her bir biti için döngü kurduk.
 - Topla -> Tek mi/Çift mi Bak -> Kaydır .
- **Kritik Dokunuş:** Eğer ara toplam tek sayı ise, Modül (N) ekleyerek sayıyı çift yaptık. Neden? Çünkü sayıyı 2'ye tam bölebilmek (sağa kaydırabilmek) için çift olması gerekir.
- **Sonuç:** Bölme işlemi olmadan modüler çarpma yapan bir "Motor" (`mon_pro.vhd`) elde ettik.

4. Aşama: Sistem Entegrasyonu (Orkestra Şefi)

Elimizde hızlı bir motor (`mon_pro`) ve bir algoritma (`Square and Multiply`) vardı. Bunları birleştirdik.

- **Yöntem: Master-Slave (Efendi-Köle) FSM Yapısı.**
- **Mantık:** Ana modül (`mod_exp`), `Square and Multiply` algoritmasının adımlarını yönetti. Her çarpma veya kare alma adımı, işi kendisi yapmak yerine `mon_pro` modülüne "start" sinyali gönderdi ve "done" sinyalini bekledi.
- **Fark:** İlk tasarımda işlemler tek satırda (`Result <= Result * Result`) yapılırken, burada bir iletişim protokolü (Handshake) kurduk.

5. Aşama: Domain Dönüşümü (Pre-computation & Post-computation)

En son adımda, sonucun yanlış çıktığını gördük ve bir matematiksel gerçeği düzelttik.

- **Sorun:** Montgomery motoru "Montgomery Uzayı"nda (Japonca konuşuyor) çalışırken, biz ona "Normal Uzay"dan (Türkçe) sayılar veriyorduk.
- **Teorem:** Montgomery Domain Dönüşümü.
 - Girişler: A yerine $A = A \times R \pmod{N}$ verilmeli.
 - Başlangıç Değeri: 1 yerine $1 = 1 \times R \pmod{N}$ verilmeli.
 - Çıkış: Sonuç çıkacak, bunu normale çevirmek için $MonPro(, 1)$ işlemi yapılmalı.
- **Uygulama:**
 1. Testbench'te Python ile hesaplayıp girdileri Montgomery formatına çevirdik (**Pre-computation**).
 2. Ana modülün (`mod_exp`) FSM'ine `START_FINAL` adımı ekleyerek sonucu 1 ile çarptırıp normale dönüştürdük (**Post-computation**).

Büyük Resim: Neyi Başardın?

Normalde bir bilgisayarın işlemcisini (CPU) yoracak çok ağır bir kriptografik işlemi;

1. Bölme işlemi kullanmadan,
2. Sadece toplama ve kaydırma ile,
3. Özelleşmiş bir donanım (IP Core) üzerinde,
4. Güvenli ve hızlı bir şekilde yapmayı başardın.

Ŗu anki projen, staj yaptığın Savunma Sanayi Ŗirketinin ürettiđi güvenli haberleşme cihazlarının, İHA yer kontrol istasyonlarının veya kriptolu telsizlerin içinde çalışan çip tasarımının temelidir.